JALA Academy
Assignment 14

Exception Handling in Python

Python solution reference • Code examples are organized by the assignment questions.

How to use this document

Save the code examples into a .py file and run with Python 3. Where a program asks for input, enter values when prompted. For file-handling exercises, create the named text file in the same folder first.

1–4. Runtime errors, handling and raising exceptions

```
# 1. This line raises ZeroDivisionError if run by itself:
# print(10 / 0)

# 2. Handle division by zero.
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero.")

# 3. Handle invalid input and division by zero.
try:
    numerator = float(input("Numerator: "))
    denominator = float(input("Denominator: "))
    print("Result:", numerator / denominator)
except ZeroDivisionError:
    print("Denominator cannot be zero.")
except ValueError:
    print("Please enter valid numbers.")

# 4. Raise an exception manually.
def check_age(age):
    if age < 18:
        raise ValueError("Age must be at least 18.")
    return "Eligible"

try:
    print(check_age(16))
except ValueError as error:
    print(error)
```

5–9. Raising, custom exception, finally, missing file and TypeError

```
def always_fails():
    raise RuntimeError("This function always raises an exception.")

try:
    always_fails()
except RuntimeError as error:
    print("Handled:", error)

class InsufficientBalanceError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError("Insufficient balance")
    return balance - amount

try:
    print(withdraw(100, 150))
except InsufficientBalanceError as error:
    print(error)

file = None
try:
    file = open("sample.txt", "r", encoding="utf-8")
```

```
    print(file.read())
except FileNotFoundError:
    print("File was not found.")
finally:
    if file is not None:
        file.close()
        print("File closed.")

try:
    open("file_that_does_not_exist.txt", "r")
except FileNotFoundError as error:
    print("Missing file:", error)

try:
    print("Age: " + 21)
except TypeError as error:
    print("Type error:", error)
```

10–14. AttributeError, IndexError, else, logging and input validation

```
import logging
logging.basicConfig(level=logging.ERROR)

class Demo:
    value = 1

try:
    print(Demo().missing_attribute)
except AttributeError as error:
    logging.error("Attribute error: %s", error)

try:
    print([1, 2, 3][10])
except IndexError as error:
    logging.error("Index error: %s", error)

try:
    result = 10 / 2
except ZeroDivisionError:
    print("Division failed")
else:
    print("No exception; result:", result)

def get_integer():
    while True:
        try:
            return int(input("Enter a whole number: "))
        except ValueError:
            print("Invalid input. Try again.")

# Uncomment to run interactive validation:
# print("Valid number:", get_integer())
```